

Intermediate code generation (ICG) bridges the front-end and back-end of a compiler, producing machine-independent code that is easy to optimize.

- If a compiler translates the source language to its target machine language without having the option for generating intermediate code, then for each new machine, a full native compiler is required.
- Intermediate code eliminates the need of a new full compiler for every unique machine by keeping the analysis portion same for all the compilers.
- The second part of compiler, synthesis, is changed according to the target machine. It becomes easier to apply the source code modifications to improve code performance by applying code optimization techniques on the intermediate code.

Intermediate Representation Intermediate codes can be represented in a variety of ways and they have their own benefits.

Graphical Representations

These represent code as nodes and edges.

- **Abstract Syntax Tree (AST)**

Hierarchical tree where interior nodes are operators, leaves are operands.

Example: $a + b * c \rightarrow$ root + with left a and right subtree for *.

- **Directed Acyclic Graph (DAG)**

Like AST but with sharing of common subexpressions (no cycles).

Advantage: Detects common subexpressions for optimization.

Stack-Based (Postfix) Representations

Resembles reverse Polish notation, used in stack machines (e.g., Java bytecode).

- **Zero-Address Code**

Operations implicitly use top of stack.

Example: $a = b + c \rightarrow$ push b, push c, add, pop a

Linear (Three-Address Code) Representations

Three-Address Code is a linear intermediate representation where each instruction contains at most three operands (two sources, one destination). It facilitates optimization and code generation by using temporary variables (t1, t2, ...) to store intermediate results.

result = operand1 operator operand2

$a = b * c + d / e$

$t1 = b * c$
 $t2 = d / e$
 $t3 = t1 + t2$
 $a = t3$

Common ways to represent TAC include Quadruples, Triples, and Indirect Triples, which store the instructions differently to manage temporaries and facilitate code optimization.

A quadruple is a record structure with four fields: op (operator), arg1 (argument 1), arg2 (argument 2), and result (result of the operation). Easy to rearrange code for optimization; explicit temporary names are stored. Consumes more memory by storing temporary variable names explicitly.

Triples: To save space, triples avoid using temporary variable names by referring to the result of a previous operation by its position (index) in the instruction list. Pros: More space-efficient than quadruples. Harder to move code for optimization, as changing one instruction's position requires updating all references to it.

Indirect Triples: This method uses a separate table to list the pointers to the triples, allowing for easy reordering of instructions without changing the actual triples. Combines the space efficiency of triples with the ease of reordering in quadruples. Uses a table of pointers (e.g., 0->100, 1->101) which point to the actual triple locations (100, 101, etc.)

#	op	arg1	arg2	Result
1	*	b	c	t1
2	/	d	e	t2
3	+	t1	t2	t3
4	=	t3		A

Quadruple

#	op	arg1	arg2
1	*	b	C
2	/	d	E
3	+	(1)	(2)
4	=	a	(3)

Triples

(0)	(100)
(1)	(101)
(2)	(102)
(3)	(103)
(4)	(104)

#	op	arg1	arg2
(100)	*	b	c
(101)	/	d	e
(102)	+	(1)	(2)
(103)	=	a	(3)

Indirect triples

Write three address code for the expression

$a = b * -c + b * -c.$

Represent the expression $A = (B + C) - (D - E)$ using 3AC, Quadruple and Triple.

Represent the instruction $A = B + C - D * E + G$ using quadruple and triple.

Write three address code for the expression

if ($x > y$)

$\text{max} = x$;

else

$\text{max} = y$;

Solution:

if $x > y$ goto L1 // Test directly and branch

$\text{max} = y$ // Else body

goto L2 // Skip then body

L1: $\text{max} = x$ // Then body

L2: // Continue

Write the three-address code for the expression

$A[i] = A[j] + 5$;

$t1 = i * 4$

$t2 = j * 4$

$t3 = A[t2]$

$t4 = t3 + 5$

$A[t1] = t4$

Write the three-address code for the expression

int $A[10][20]$;

$A[i][j] = A[x][y] + 5$;

Solution:

// Load $A[x][y]$

$t1 = x * 20$ // row offset = $i * \text{columns}$

$t2 = t1 + y$ // add column offset

$t3 = t2 * 4$ // multiply by element size (4 bytes)

$t4 = A + t3$ // base address + offset

$t5 = *t4$ // load value $A[x][y]$

// Compute new value

$t6 = t5 + 5$ // $A[x][y] + 5$

// Store to $A[i][j]$

$t7 = i * 20$ // row offset

$t8 = t7 + j$ // add column offset

$t9 = t8 * 4$ // multiply by element size

$t10 = A + t9$ // base address + offset

```
t10 = t6          // store to A[i][j]
```

Boolean expressions

Boolean expressions have two primary purposes. They are used to compute logical values, but more often they are used as conditional expressions in statements that alter the flow of control, such as if-then-else, or while-do statements.

Boolean expressions are composed of the boolean operators (and, or, and not) applied to elements that are boolean variables or relational expressions.

Methods of Translating Boolean Expressions:

There are two principal methods of representing the value of a boolean expression. They are:

- * To encode true and false numerically and to evaluate a boolean expression analogously to an arithmetic expression. Often, 1 is used to denote true and 0 to denote false.
- * To implement boolean expressions by flow of control, that is, representing the value of a boolean expression by a position reached in a program. This method is particularly convenient in implementing the boolean expressions in flow-of-control statements, such as the if-then and while-do statements.

Numerical Representation

Here, 1 denotes true and 0 denotes false. Expressions will be evaluated completely from left to right, in a manner similar to arithmetic expressions.

For example:

- * The translation for a or b and not c is the three-address sequence

```
t1: = not c
```

```
t2: = b and t1
```

```
t3: = a or t2
```

- * A relational expression such as $a < b$ is equivalent to the conditional statement
- * if $a < b$ then 1 else 0

which can be translated into the three-address code sequence (statement numbers at 100) :

```
100: if a < b goto 103
```

```
101: t: = 0
```

```
102: goto 104
```

```
103: t: = 1
```

```
104:
```

Translation scheme using a numerical representation for booleans

Production	Semantic Rule
$E \rightarrow E_1 \text{ or } E_2$	{E.place = newtemp(); gen(E. place "=" E ₁ .place "or" E ₂ .place)}
$E \rightarrow E_1 \text{ and } E_2$	{E.place = newtemp(); gen(E. place "=" E ₁ .place "and" E ₂ .place)} {E.place = newtemp();
$E \rightarrow \text{not } E_1$	gen(E. place "=" "not" E ₁ .place)}
$E \rightarrow (E_1)$	{E.place = E ₁ .place}
$E \rightarrow \text{id}_1 \text{ relop id}_2$	{E.place = newtemp(); gen("if" id ₁ .place relop id ₂ .place "goto" nextstat + 3) gen(E.place "=" "0") gen("goto" nextstat + 2) gen(E. place "=" "1")}
$E \rightarrow \text{true}$	{E.place = newtemp(); gen(E. place "=" "1")}
$E \rightarrow \text{false}$	{E.place = newtemp(); gen(E.place=" "0")}

Boolean Expression: Example: Give three address code for the following: while(P < Q) do
if(R < S) then a = b + c;

Solution 1. if P < Q goto (3) 2. goto (8)

3. if RS goto (5)

4. goto (1)

5. t:= b+c

6. a: = t1

7. goto (1)

8. Next

Translate $a < b$ or $c < d$ and $e < f$ into three address code

Flow-of-Control Statements: We now consider the translation of boolean expressions into three address code in the context of if-then, if-then-else, and while-do statements such as those generated by the following grammar:

S → if E then S₁

| if E then S₁ else S₂

| while E do S₁

In each of these productions, E is the Boolean expression to be translated. In the translation, we assume that a three-address statement can be symbolically labeled, and that the function newlabel returns a new symbolic label each time it is called.

* E.true is the label to which control flows if E is true, and E.false is the label to which control flows if E is false.

* The semantic rules for translating a flow-of-control statement S allow control to flow from the translation S.code to the three-address instruction immediately following S.code.

* S.next is a label that is attached to the first three-address instruction to be executed after the code for S.

Production	Semantic Rule
S → if E then S ₁	{E.true = newlabel(); E.false = S.next; S ₁ .next = S.next; S.code = E.code gen(E.true, ":") S ₁ .code}
S → if E then S ₁ else S ₂	{E.true = newlabel(); E.false = newlabel(); S ₁ .next = S.next; S ₂ .next = S.next; S.code = E.code gen(E.true, ":") S ₁ .code gen(" GOTO", S.next) gen(E.false, " :") S ₂ .code}
S → while E do S ₁	{S.begin = newlabel(); E.true = newlabel(); E.false = S.next; S ₁ .next = S.next; S.code = gen(S.begin ":") E.code gen(E.true, ":") S ₁ .code gen("GOTO", S.begin)}

Example: Give three address code for the following: While (a < 5) do a = b + 2

Solution: -

```

L1:
If a < 5 goto L2
goto last
L2:
t1 = b+2
a = t1 goto L1
last:

```

Example: Give three address code for the following:

<pre> while a<b do if c<d then else x = y + z x=y-z </pre>	<pre> Solution: - L1. if a<b then GOTO L2 GOTO LNEXT L2. if c<d then GOTO L3 GOTO L4 L3. t1 = y + z x = t1 GOTO Li L4. t1=y - z x = t1 GOTO Li LNEXT. </pre>
--	--

Case Statement

The case statement is a conditional statement used to execute different blocks of code based on the evaluated value of the input expression. It is available in various programming languages, and the syntax varies depending on the language. The switch-statement syntax is as shown below:

```

switch (expression)
begin
    case value1: statement1
    case value2: statement2
    ...
    case valuen: statementn
    default: statement_default
end

```

The intended translation of a switch is code to:

1. Evaluate the selector expression
2. Find matching case value
3. Execute corresponding statement
4. Handle fall-through semantics (if no explicit break)
5. Efficient dispatch for different case distributions

Implementation Strategies of case statements

1. Sequential If-Goto (for small cases)

if t = V1 goto L1

if t = V2 goto L2

...

goto Ln (default)

2. Table Lookup Method

Store (value, label) pairs in a table

Compare using a loop

Better than sequential when cases increase

3. Hash Table

Use hashing for faster lookup

Efficient for large and scattered values

4. Jump Table (Best Case)

Used when values are:

Dense (close range like 1–10)

Syntax-Directed Translation of Case Statements

Now let's understand how the compiler converts switch into **intermediate code (3-address code)**.

Given Switch Statement	Generated Intermediate Code
switch E	t = evaluate(E)
begin	goto test
case V1 : S1	L1: code for S1
case V2 : S2	goto next
...	L2: code for S2
default : Sn	goto next

end	... Ln: code for default Sn goto next test: if t = V1 goto L1 if t = V2 goto L2 ... goto Ln next:
-----	---

Step-by-Step Translation Process

1. When switch is seen: • Create: - Temporary variable $\rightarrow t$ - Labels $\rightarrow$ test, next	4. For Default Case Ln: code for Sn goto next
2. Evaluate Expression t = E goto test	5. Generate Test Code (Important Part) test: if t = V1 goto L1 if t = V2 goto L2 ... goto Ln (default)
3. For Each Case • Create label Li • Store (Vi, Li) in a stack • Generate: Li: code for Si goto next	6. Exit Label next:

Backpatching

Back patching is a technique to solve the problem of replacing symbolic names into the goto statements by the actual target addresses. Some languages do not permit to use symbolic names in the branches, for this we maintain a list of branches that have the same target labels and then

replace them once they are defined. To manipulate the lists of jumps. To perform backpatching the following three functions are used:

makelist(i): This function creates a new list containing an index i into the array of statements and then returns a pointer pointing to the newly created list.

merge(p₁, p₂): This function concatenates the two lists pointed to by the pointers p₁ and p₂ respectively, and then returns a pointer to the concatenated list.

backpatch(p, i): This function inserts i as the target labels for each of the statements on the list pointed by p.

Backpatching for Boolean Expressions

Production	Semantic Rule
$E \rightarrow E1 \parallel M E2$	backpatch(E1.falselist, M.instr) E.truelist = merge(E1.truelist, E2.truelist) E.falselist = E2.falselist
$E \rightarrow E1 \&\& M E2$	backpatch(E1.truelist, M.instr) E.truelist = E2.truelist E.falselist = merge(E1.falselist, E2.falselist)
$E \rightarrow \text{not } E1$	E.truelist = E1.falselist E.falselist = E1.truelist
$E \rightarrow (E1)$	E.truelist = E1.truelist E.falselist = E1.falselist
$E \rightarrow \text{id1 relop id2}$	E.truelist = makelist(nextinstr) E.falselist = makelist(nextinstr+1) Generate: if id1 relop id2 goto _ Generate: goto _
$M \rightarrow \epsilon$	M.instr = nextinstr
$E \rightarrow \text{true}$	E.truelist = makelist(nextinstr); emit('goto_');
$B \rightarrow \text{false}$	B.falselist = makelist(nextinstr); emit('goto_');

Annotated parse tree for $x < 100 \parallel x > 200 \&\& x \neq y$

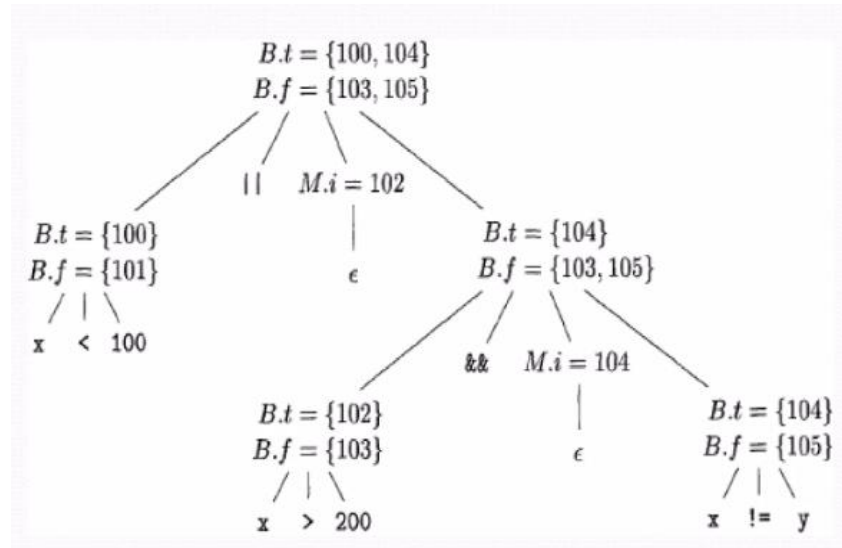
100 if x < 100 then 106

101 goto 102

```

102 if x>200 then 104
103 goto 107
104 if x != y then 106
105 goto 107
106 true
107 false

```



Procedure calls

Procedure calls in three-address code (TAC) are implemented by evaluating arguments, passing them via param statements, and calling the function using call p, n (where p is the procedure and n is the number of arguments). This mechanism handles function execution, parameter passing, and return values, allowing for stack-based management, parameter evaluation, and flow control during compilation.

Key Components of Procedure Calls in TAC

- **param Statement:** Used to pass parameters. Each parameter $x_1, x_2, \dots, x_n$ is passed using param x_i statements.
- **call Statement:** call p, n is used to call a procedure p with n parameters.
- **Return Value:** The return value, if any, is typically stored in a temporary variable, for example $y = \text{call } p, n$.
- **Activation Record Handling:** Space is allocated for activation records, and control transfers from the calling procedure to the called procedure.

General Procedure Call Syntax

A function call $P(x_1, x_2, \dots, x_n)$ with n parameters is represented as:

1. param x_1
2. param x_2
3. ...
4. param x_n
5. call p, n

Example: Calling a Function

Consider a function `int add (int a, int b)`. The call `result = add (x, y)`; can be represented in three-address code as:

```
param x      // Pass argument x
```

```
param y      // Pass argument y
```

```
t1 = call add, 2 // Call procedure 'add' with 2 parameters
```

```
result = t1    // Store the returned value
```